APCS 程式設計系列 · U12 · 實作高級 · C++ / Python 對照版

圖論與樹 詳解

鄰接矩陣與鄰接串列 · BFS 最短步數 · DFS 與連通塊 · 樹的深度與子樹 · 前中後序 · 樹直徑 · 拓撲排序

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
A	A	D	C	C	B	D	A	D	B
11	**12**	**13**	**14**	**15**	**16**				
C	B	D	A	C	B				

Part A 程式識讀詳解

第 1 題 答案：(A)

- 鄰接矩陣第 i 列的總和，就是點 i 的度數（有幾個鄰居）。
- 四個點的度數依序是 2、3、2、1，最大的是點 1，度數 3。

第 2 題 答案：(A)

- 每條邊只存了 a[u][v] 一個方向，沒有存 a[v][u]。
- 從點 1 出發只看得到 a[1][0]，走到點 0；點 0 那一列全是 0，走不下去。共走到 1、0 兩個點。

【錯】以為五個點都連在一起就選 5。無向圖少加一次邊，就變成單行道。

第 3 題 答案：(D)

- 鄰接矩陣是 $n \times n = 10^{10}$ 格，以 int 計約 40 GB，不可能放得下。
- 鄰接串列只存真的存在的邊：無向圖每條邊存兩次，共 4×10^5 個數字。

第 4 題 答案：(C)

- 點 0 的鄰居是 1、3，依編號放進佇列：佇列 = 1, 3。
- 拿出 1：鄰居 4 沒走過，放進去；拿出 3：鄰居 2 沒走過，放進去（4 已經走過）。
- 再依序拿出 4、2。輸出 0 1 3 4 2：一層一層往外。

【錯】0 1 4 3 2 是 DFS（一路走到底）的順序。

第 5 題 答案：(C)

- d 是「離點 0 幾步」。點 0 的鄰居 1、2：d = 1。
- 點 1 帶出 3（d = 2），點 2 帶出 5（d = 2，3 已經有值）；點 3 帶出 4（d = 3）。
- 所以 d[3]、d[4]、d[5] = 2 3 2。BFS 第一次碰到就是最短步數。

第 6 題 答案：(B)

- dfs(0) 先找到鄰居 1，馬上進 dfs(1)；1 的鄰居 4 沒走過，進 dfs(4)。
- 4 的鄰居 3 沒走過，進 dfs(3)；3 的鄰居 2 沒走過，進 dfs(2)。
- 一路走到底：0 1 4 3 2。

【錯】0 1 3 4 2 是 BFS 的順序（同一張圖，上上一題考過）。

第 7 題 答案：(D)

- 沒有 visited，從 u 走到 v 之後，v 又會走回 u，u 再走到 v……永遠不會結束。
- BFS 也一樣：沒標記的點會被一直放進佇列。

第 8 題 答案：(A)

- 每呼叫一層遞迴都要佔用一些堆疊空間，10^6 層很可能超過系統給的堆疊大小。
- 改成自己用陣列（vector）當堆疊，或改用 BFS，就沒有深度問題。

第 9 題 答案：(D)

- Python 預設遞迴上限約 1000 層，長鏈要遞迴 10^5 層，會丟出 RecursionError。
- sys.setrecursionlimit 可以調高，但太深仍可能直接崩潰；最安全是改用 list 當堆疊或用 BFS。

第 10 題 答案：(B)

- 樹的定義：連通而且沒有環；n 個點的樹剛好有 n - 1 條邊。
- 葉子的度數是 1；任兩點之間只有唯一一條路徑（有兩條就形成環了）。

第 11 題 答案：(C)

- d 是深度：2、3 是 1；4、5、6、8 是 2；7 的父親是 5，深度 3。最大深度 3。
- c 是小孩數：1、2、3、5 有小孩；沒有小孩的 4、6、7、8 是葉子，共 4 個。

第 12 題 答案：(B)

- sz 是子樹大小。這棵樹每個父節點的編號都比小孩小，從 8 倒著加回去，加到 i 時它的小孩都已經算完了。
- 2 的子樹是 {2, 4, 5, 7}：4 個；3 的子樹是 {3, 6, 8}：3 個；根的子樹是全部 8 個。

【錯】忘了 7 掛在 5 底下、會一路加到 2，就會把 sz[2] 算成 3。

第 13 題 答案：(D)

- 先走左、再印自己、最後走右：這是中序走訪。
- 樹的樣子：1 的小孩是 2、3；2 只有右小孩 4；4 只有左小孩 7；3 的小孩是 5、6。
- 中序：左子樹 2 7 4，根 1，右子樹 5 3 6，合起來 2 7 4 1 5 3 6。

【錯】4 是 2 的「右」小孩，7 是 4 的「左」小孩，所以 7 要排在 4 前面。

第 14 題 答案：(A)

- 前序第一個 A 是根；中序裡 A 左邊的 D B E 是左子樹，右邊的 C F 是右子樹。
- 左子樹前序 B D E：B 是根、D 在左、E 在右。右子樹前序 C F：C 是根，中序 C 在 F 前面，所以 F 是 C 的右小孩。
- 後序（左、右、根）：D E B、F C、最後 A。

第 15 題 答案：(C)

- 這就是兩次 BFS 求樹直徑：第一次找到的 a 一定是某條最長路徑的端點。
- 從端點 a 出發再找最遠點，距離就是直徑。只有樹是一條鏈時，直徑才剛好是點數減 1。

第 16 題 答案：(B)

- deg 是入度：0 有 2→0、3→0 兩條，入度 2；1 入度 1；4 入度 2；2、3 入度 0，先進佇列。
- 拿出 2：0 的入度變 1。拿出 3：0 變 0、1 變 0，依序放進 0、1。
- 拿出 0：4 變 1；拿出 1：4 變 0，放進 4。順序 2 3 0 1 4。

Part B 程式實作參考解答

實作 1 機器人走迷宮

解題思路

- 每一格是一個點，上下左右相鄰的走道之間有邊，每條邊都是 1 步：典型的 BFS 最短步數。
- 先掃一遍地圖找出 S、E 的位置；d 陣列全設 -1，從 S 開始 BFS，第一次走到某格的步數就是最短。
- BFS 結束後，d[E] 還是 -1 就代表走不到。

【錯】起點不一定在左上角，終點也不一定在右下角，要自己找。

【錯】先檢查有沒有出界，再看那一格是不是 #；而且「放進佇列時」就要標記，不然同一格會重複進佇列。

【錯】Python 用 list 的 pop(0) 當佇列會超時，要用 deque。

C++ 機器人走迷宮

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    int sr = 0, sc = 0, er = 0, ec = 0;
    for (int r = 0; r < n; r++) {
        cin >> g[r];
        for (int c = 0; c < m; c++) {
            if (g[r][c] == 'S') sr = r, sc = c;
            if (g[r][c] == 'E') er = r, ec = c;
        }
    }
    vector<vector<int>> d(n, vector<int>(m, -1));
    int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    queue<pair<int, int>> q;
    d[sr][sc] = 0;
    q.push({sr, sc});
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            if (g[nr][nc] == '#' || d[nr][nc] != -1) continue;
            d[nr][nc] = d[r][c] + 1;
            q.push({nr, nc});
        }
    }
    cout << d[er][ec] << "\n";
}
```

Python

```python
import sys
from collections import deque

data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = data[2:2 + n]
for r in range(n):
    for c in range(m):
        if g[r][c] == "S":
            sr, sc = r, c
        elif g[r][c] == "E":
            er, ec = r, c
D = ((-1, 0), (1, 0), (0, -1), (0, 1))
d = [[-1] * m for _ in range(n)]
d[sr][sc] = 0
q = deque([(sr, sc)])
while q:
    r, c = q.popleft()
    for dr, dc in D:
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < m:
            if g[nr][nc] != "#" and d[nr][nc] == -1:
                d[nr][nc] = d[r][c] + 1
                q.append((nr, nc))
print(d[er][ec])
```

時間複雜度：O(nm)：每一格最多進佇列一次

實作 2 數島嶼

解題思路

- 每一格陸地是一個點，上下左右相鄰的陸地之間有邊：島嶼數量就是連通塊的數量。
- 由上到下、由左到右掃每一格；碰到還沒走過的陸地，島數加1，再用DFS（或BFS）把整座島走完、順便數面積。
- 標記走過的小技巧：走過的陸地直接改成「.」（把島沉掉），就不用另外開 visited 陣列。

【錯】忘記標記走過的格子：同一座島會被重複計算，或在島上繞圈繞不完。

【錯】整張地圖可能是一座蛇形大島，遞迴 DFS 深到十幾萬層：C++ 可能堆疊溢位、Python 一定超過遞迴上限。

C++ 數島嶼

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    int islands = 0, best = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (g[i][j] != '#') continue;
            islands++;
            int area = 0;
            g[i][j] = '.';                  // 沉島：標記走過
            vector<pair<int, int>> st = {{i, j}};
            while (!st.empty()) {
                auto [r, c] = st.back();
                st.pop_back();
                area++;
                for (int k = 0; k < 4; k++) {
                    int nr = r + dr[k], nc = c + dc[k];
                    if (nr < 0 || nr >= n) continue;
                    if (nc < 0 || nc >= m) continue;
                    if (g[nr][nc] == '#') {
                        g[nr][nc] = '.';
                        st.push_back({nr, nc});
                    }
                }
            }
            best = max(best, area);
        }
    cout << islands << " " << best << "\n";
}
```

Python

```python
import sys

data = sys.stdin.read().split()
n, m = int(data[0]), int(data[1])
g = [list(row) for row in data[2:2 + n]]
D = ((-1, 0), (1, 0), (0, -1), (0, 1))
islands = best = 0
for i in range(n):
    for j in range(m):
        if g[i][j] != "#":
            continue
        islands += 1
        area = 0
        g[i][j] = "."               # 沉島：標記走過
        st = [(i, j)]
        while st:
            r, c = st.pop()
            area += 1
            for dr, dc in D:
                nr, nc = r + dr, c + dc
                if 0 <= nr < n and 0 <= nc < m:
                    if g[nr][nc] == "#":
                        g[nr][nc] = "."
                        st.append((nr, nc))
        best = max(best, area)
print(islands, best)
```

時間複雜度：O(nm)：每一格最多被放進堆疊一次

實作 3 家族樹統計

解題思路

- 邊沒有方向，就當無向邊存，從根 1 做 BFS：第一次走到 v 時，v 的深度 = u 的深度 + 1。
- 處理 u 時，數一數有幾個鄰居是第一次走到的，那些就是 u 的小孩；小孩數是 0 就是葉子。
- 高度取深度最大值，深度總和一路累加。

【錯】用「度數 = 1」判斷葉子：根只有一個小孩時也會被算進去（範例 2 的答案是 1，不是 2）。n = 1 時根本身就是葉子。

【錯】長鏈時：C++ 的深度總和約 5×10^9 要用 long long；Python 遞迴會超過上限，用 BFS。

C++ 家族樹統計

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> dep(n + 1, -1), q = {1};
    dep[1] = 0;
    int height = 0, leaves = 0;
    long long total = 0;
    for (int h = 0; h < (int)q.size(); h++) {
        int u = q[h], kids = 0;
        height = max(height, dep[u]);
        total += dep[u];
        for (int v : adj[u])
            if (dep[v] == -1) {
                dep[v] = dep[u] + 1;
                q.push_back(v);
                kids++;
            }
        if (kids == 0) leaves++;
    }
    cout << height << " " << leaves << " " << total << "\n";
}
```

Python

```python
import sys

data = sys.stdin.read().split()
n = int(data[0])
adj = [[] for _ in range(n + 1)]
for i in range(n - 1):
    u, v = int(data[1 + 2 * i]), int(data[2 + 2 * i])
    adj[u].append(v)
    adj[v].append(u)
dep = [-1] * (n + 1)
dep[1] = 0
q = [1]
h = height = leaves = total = 0
while h < len(q):
    u = q[h]
    h += 1
    height = max(height, dep[u])
    total += dep[u]
    kids = 0
    for v in adj[u]:
        if dep[v] == -1:
            dep[v] = dep[u] + 1
            q.append(v)
            kids += 1
    if kids == 0:
        leaves += 1
print(height, leaves, total)
```

時間複雜度：O(n)

實作 4 最遠的兩個村莊

解題思路

- 樹直徑：從任一點出發找最遠點 a，再從 a 出發找最遠點 b，a 到 b 的距離就是答案。
- 邊有長度也一樣：從起點走訪整棵樹，走到 v 時距離 = 到 u 的距離 + 這條邊的長度。樹上兩點只有一條路，走訪順序不影響距離。
- 兩次走訪都是 O(n)，不能對每個點各做一次（那是 $O(n^2)$）。

【錯】C++ 的距離最大約 10^{14}，要用 long long。

【錯】只做一次走訪、從點 1 找最遠點就輸出：點 1 不一定在直徑的端點上。例如 4-3-2-1 一條鏈再接 1-5-6（後兩條長 2），從點 1 最遠只有 4，直徑卻是 7。

【錯】一條長鏈有 10^5 層：遞迴 DFS 在兩種語言都可能出事，用 BFS 最安全。

C++ 最遠的兩個村莊

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<pair<int, int>>> adj;
vector<long long> d;

int farthest(int s) {              // 回傳離 s 最遠的點，d 存距離
    d.assign(n + 1, -1);
    d[s] = 0;
    vector<int> q = {s};
    int best = s;
    for (int h = 0; h < (int)q.size(); h++) {
        int u = q[h];
        if (d[u] > d[best]) best = u;
        for (auto [v, w] : adj[u])
            if (d[v] == -1) {
                d[v] = d[u] + w;
                q.push_back(v);
            }
    }
    return best;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    adj.assign(n + 1, {});
    for (int i = 0; i < n - 1; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    int a = farthest(1);
    int b = farthest(a);
    cout << d[b] << "\n";
}
```

Python

```python
import sys

def farthest(s):
    d = [-1] * (n + 1)
    d[s] = 0
    q, h, best = [s], 0, s
    while h < len(q):
        u = q[h]
        h += 1
        if d[u] > d[best]:
            best = u
        for v, w in adj[u]:
            if d[v] == -1:
                d[v] = d[u] + w
                q.append(v)
    return best, d[best]

data = sys.stdin.read().split()
n = int(data[0])
adj = [[] for _ in range(n + 1)]
for i in range(n - 1):
    u, v, w = map(int, data[1 + 3 * i:4 + 3 * i])
    adj[u].append((v, w))
    adj[v].append((u, w))
a, _ = farthest(1)
b, ans = farthest(a)
print(ans)
```

時間複雜度：O(n)：兩次走訪

實作 5 切一刀分兩隊

解題思路

- 以 1 為根。剪斷「v 和它父親」之間的邊，v 那一邊剛好是 v 的子樹：sz[v] 人，另一邊 n - sz[v] 人。
- 所以只要算出每個點的子樹大小（樹上 DP）：從根 BFS 記下順序，倒過來把 sz[v] 加到父親身上。
- 除了根以外，每個點 v 對應一條邊，人數差是 $|n - 2 \times sz[v]|$，取最小值並計數。

【錯】每剪一條邊都重新走訪一次算人數，是 $O(n^2)$，$n = 10^5$ 會超時。

【錯】根沒有父親，不對應任何邊，不能算進去。

【錯】一條長鏈有 10^5 層：用遞迴算子樹大小兩種語言都危險，用 BFS 順序倒著做。

C++ 切一刀分兩隊

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> par(n + 1, 0), sz(n + 1, 1), q = {1};
    for (int h = 0; h < (int)q.size(); h++) {
        int u = q[h];
        for (int v : adj[u])
            if (v != par[u]) {
                par[v] = u;
                q.push_back(v);
            }
    }
    for (int i = n - 1; i >= 1; i--)      // 由下往上累加子樹大小
        sz[par[q[i]]] += sz[q[i]];
    int best = n, cnt = 0;
    for (int v = 2; v <= n; v++) {        // 每個非根的點對應一條邊
        int diff = abs(n - 2 * sz[v]);
        if (diff < best) best = diff, cnt = 1;
        else if (diff == best) cnt++;
    }
    cout << best << " " << cnt << "\n";
}
```

Python

```python
import sys

data = sys.stdin.read().split()
n = int(data[0])
adj = [[] for _ in range(n + 1)]
for i in range(n - 1):
    u, v = int(data[1 + 2 * i]), int(data[2 + 2 * i])
    adj[u].append(v)
    adj[v].append(u)
par, sz = [0] * (n + 1), [1] * (n + 1)
q, h = [1], 0
while h < len(q):
    u = q[h]
    h += 1
    for v in adj[u]:
        if v != par[u]:
            par[v] = u
            q.append(v)
for u in reversed(q[1:]):          # 由下往上累加子樹大小
    sz[par[u]] += sz[u]
best, cnt = n, 0
for v in range(2, n + 1):          # 每個非根的點對應一條邊
    diff = abs(n - 2 * sz[v])
    if diff < best:
        best, cnt = diff, 1
    elif diff == best:
        cnt += 1
print(best, cnt)
```

時間複雜度：O(n)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
圖的表示	A1、A2、A3		綠 黃 紅	講義第 1 段（鄰接矩陣與鄰接串列、無向圖加兩次）
BFS 與最短步數	A4、A5、B1		綠 黃 紅	講義第 2 段
DFS 與 visited	A6、A7、A8、A9、B2		綠 黃 紅	講義第 3 段（含遞迴太深）
樹的基本性質	A10、A11、B3		綠 黃 紅	講義第 4 段
樹的走訪	A13、A14		綠 黃 紅	講義第 5 段
樹直徑與樹上 DP	A12、A15、B4、B5		綠 黃 紅	講義第 4 段（子樹大小）、第 6 段
拓撲排序	A16		綠 黃 紅	講義第 7 段（選修）